AUTOMATAS FINITOS

Los autómatas finitos sirven para describir lenguajes regulares. Según su forma de funcionamiento se dividen en:

- ⇒ **Determinístico (AFD):** dada una entrada, existe un **único estado** al que se puede llegar
- ⇒ **No determinístico (AFND):** dada una entrada, el AF puede mover a más de un estado a la vez
- ⇒ **No determinístico (AFND-λ):** puede tener transiciones vacías (sin consumir entrada)

☞ AUTOMATAS FINITOS DETERMINISTICOS

Un **AFD** es $M = \langle Q, A, \delta, q_0, F \rangle$

Q Conjunto finito de estados

A Alfabeto

δ: **Q x A → Q** Función de transición

$q_0 \in$ **Q** Estado Inicial

F ⊆ Q Conjunto de Estados de Aceptacion

<u>Extensión de δ para cadenas:</u> $\delta^*: Q \times A^* \to Q$

$\delta^*(q, \lambda) = q$

$\forall\, \omega \in A^*,\ a \in A,\ \delta^*(q, a\omega) = \delta^*(\delta(q,a),\omega)$

Lenguaje aceptado por un AFD $L(M) = \{ \omega \in A^* \ /\ \delta^*(q_0, \omega) \in F \}$

Dos AF M y M' se dicen **equivalentes** ($M \equiv M'$) si reconocen el mismo lenguaje, es decir, si $L(M) = L(M')$

☞ AUTOMATAS FINITOS NO DETERMINISTICOS

Un **AFND** es $M = \langle Q, A, \delta, q_0, F \rangle$

Q Conjunto finito de estados

A Alfabeto

$\delta \subseteq$ **Q x A x Q** Función (o relación) de transición ($\delta : Q \times A \to \mathcal{P}(Q)$)

$q_0 \in$ **Q** Estado Inicial

F ⊆ Q Conjunto de Estados de Aceptación.

Lenguaje aceptado por un AFND $L(M) = \{ \omega \in A^* \ /\ \delta^*(q_0, \omega) \cap F \neq \varnothing \}$

Transformación de AFND a AFD

Todo AFND puede convertirse en un AFD equivalente, esto ocasionará un incremento en el número de estados.

AFND	AFD
Q	$\mathcal{P}(Q)$
A	A
q_0	$\{ q_0 \}$
F	$\{ p \subseteq Q \ /\ p \cap F \neq \phi \}$
δ	$\delta(\{q\}, a) = \{ q' \in Q \ /\ (q, a, q') \in \delta \}\ \forall a \in A$ $\delta(\{q_1,..,q_n\}, a) = \bigcup_{i=1}^{n} \{ q' \in Q \ /\ (q_i, a, q') \in \delta \}\ \forall a \in A$

Si $M_{AFND} \cong M_{AFD} \Rightarrow L(M_{AFND}) = L(M_{AFD})$

Un **AFND-λ** es M < Q, A, δ, q_0, F >

Q Conjunto finito de estados

A Alfabeto

δ ⊆ Q x (A∪ {λ}) x Q Función (relación) de transición ($\delta : Q \times (A \cup \{\lambda\}) \rightarrow \mathcal{P}(Q)$)

q_0∈ Q Estado Inicial

F ⊆ Q Conjunto de Estados de Aceptación.

Transformación de AFND-λ a AFD

Todo AFND-λ puede convertirse en un AFD equivalente.

Clausura

Se define $V \subset Q \times Q$ (pares de estados con transiciones lambda)

- ✓ $(q,q') \in V$ si $\delta(q,\lambda) = q'$
- ✓ si $(q,q') \in V$ y $(q',q'') \in V$ entonces $(q,q'') \in V^2 \subset V^*$
- ✓ $(q,q) \in V$, $\forall q \in Q$

Clausura de un estado

$Clau(\{q\}) = \{q' \varepsilon Q / (q,q') \varepsilon V^* \}$

$Clau(\{q_1,..,q_n\}) = \bigcup_{1 \le i \le n} .clau(\{q_i\})$

AFND-λ	AFD
Q	$\mathcal{P}(Q)$
A	A
q_0	$clau(\{ q_0 \})$
F	$\{ p \subseteq Q / p \cap F \neq \phi\}$
δ	$\delta(clau(\{q\}), a) = \{q'' \in Q / (q',a, q'') \in \delta, q' \in clau(\{q\}) \} \forall a \in A$ $\delta(clau(\{q_1,..,q_n\}), a) = \bigcup_{i=1}^{n} \{(q',a, q'') \in \delta, q' \in clau(\{q_i\}) \} \forall a \in A$

Si $M_{AFND-\lambda} \cong M_{AFD} \Rightarrow L(M_{AFND-\lambda}) = L(M_{AFD})$

Minimización de AFD

El objetivo es, dado un AFD, construir otro que reconoce el mismo lenguaje con un número mínimo de estados.

La idea básica que permite reducir el número de estados de un AFD consiste en eliminar un estado siempre que ya exista otro que realiza la misma función.

Dos estados realizan la misma función cuando cualquier palabra de entrada da el mismo resultado tanto si se parte de uno como de otro estado. Se dice que los dos estados son, en este caso, **indistinguibles**. Hay que tener en cuenta que, al referir al resultado que da una entrada a partir de un estado, nos limitamos a uno de los dos que tienen relevancia: que se acceda a un estado de aceptación o que se acceda a un estado de no aceptación.

Pueden existir varios estados que sean indistinguibles de uno dado. Es suficiente, conservar un solo estado para cada <u>clase de estados indistinguibles</u>.

Definición Sea M = < Q, A, δ, q_0, F > un AFD. Dados q, p ∈ Q diremos que p es equivalente a q, o que p y q son **indistinguibles** (p ≅ q) así:

$p \cong q \Leftrightarrow \forall x \in A^* (\delta^*(p, x) \in F \Leftrightarrow \delta^*(q, x) \in F)$

o

$p \cong q \Leftrightarrow \forall x \in A^* (\delta^*(p, x) \notin F \Leftrightarrow \delta^*(q, x) \notin F)$

$p \ncong q \Leftrightarrow \exists x \in A^* (\delta^*(p, x) \notin F$ y $\delta^*(q, x) \in F)$

o

$p \ncong q \Leftrightarrow \exists x \in A^* (\delta^*(p, x) \in F$ y $\delta^*(q, x) \notin F)$

dado que

$\delta^*(p, a\beta)=\delta^*(\delta(p, a),\beta)$

$\delta^*(q, a\beta) =\delta^*(\delta(q, a),\beta)$

$\Rightarrow p \ncong q \Leftrightarrow \delta(p, a) \neq \delta(q, a)$

$q \cong_0 q' \Leftrightarrow (q \in F$ y $q' \in F)$ o $(q \notin F$ y $q' \notin F)$

$q \cong_{k+1} q' \Leftrightarrow q \cong_k q'$ y $\delta(q, a) \cong \delta(q', a) \quad \forall a \in A$

Algoritmo

1) $\pi_0 = \{Q\text{-}F, F\}$
2) Obtener π_{k+1} a partir de π_k
   $B(p, \pi_{k+1}) = B(q, \pi_{k+1}) \Leftrightarrow B(p, \pi_k) = B(q, \pi_k) \wedge \forall a \in A, B(\delta(p, a), \pi_k) = B(\delta(q, a), \pi_k)$
3) Si π_{k+1} es distinta a π_k ir al paso **2)** sino terminar